

Metodología con enfoque Spec-Driven y de simulación para el desarrollo de sistemas ciberfísicos

Modelo de procesos en SPEM 2.0 para un sistema de riego autónomo guiado por drones en caficultura

Autor 1, Autor 2, Autor 3, Autor 4

Programa de Ingeniería · Universidad

« reemplazar por los nombres y códigos del equipo »

Resumen— Los sistemas ciberfísicos exigen coordinar, sobre un mismo cronograma, el desarrollo de software y la construcción de hardware que aún no existe. Este trabajo propone un modelo de procesos en SPEM 2.0 para un sistema de riego autónomo guiado por drones en caficultura. El modelo parte de la metodología con enfoque BDD y MDD para proyectos IoT y conserva su armazón —ciclos cortos, colaboración multidisciplinar y artefactos como eje de la comunicación—, pero sustituye sus dos instrumentos: el Spec-Driven Development ocupa el lugar de BDD y los entornos de simulación SIL, HIL y el gemelo digital ocupan el de MDD. El resultado son cuatro Fases, veintiuna Tareas y ocho Roles, organizados en una Iteración 0 de arranque de catorce semanas y un ciclo de crecimiento permanente de incrementos de dos a cuatro semanas, gobernado por cuatro puertas de decisión. Dos de esas puertas las cierra el caficultor, que en este modelo no es un interesado externo sino un Rol que ejecuta Tareas y responde por artefactos.

Palabras clave— *SPEM 2.0, modelo de procesos, sistema ciberfísico, Spec-Driven Development, simulación SIL/HIL, gemelo digital, IoT, agricultura de precisión.*

I. INTRODUCCIÓN

El sistema que motiva este trabajo es un cafetal instrumentado: una red de sensores enterrados en el terreno, un conjunto de drones que sobrevuela los lotes y una lógica de control que decide, con esa información, cuándo y cuánto regar. No es un proyecto de software con periféricos. Es un sistema ciberfísico, y esa diferencia cambia la forma de organizar el trabajo.

La razón es que las fallas cruzan la frontera entre los dos mundos. Una regla de riego mal escrita no se manifiesta como una excepción en un log: se manifiesta como una válvula que no abre. Un sensor mal calibrado no se manifiesta como un dato raro: se manifiesta como una decisión de control equivocada que riega un lote que no lo necesitaba. Un equipo que trate el hardware como un detalle de implementación descubre estas fallas tarde, en campo, cuando corregirlas ya cuesta una temporada.

A esto se suma que buena parte del hardware no existe cuando empieza el software. Los sensores están por comprarse, la electrónica por diseñarse y el dron por integrarse. Si el equipo de software espera a que el hardware llegue, pierde meses; si avanza sin ninguna referencia, construye contra un sistema imaginario.

El punto de partida de este trabajo es la metodología con enfoque BDD y MDD para el desarrollo de proyectos IoT [1], que ataca el problema de fondo correcto: la comunicación entre los actores técnicos y los que no lo son, y el uso de mode-

los como base del desarrollo. Su armazón se conserva íntegro. Lo que se sustituye son los dos instrumentos concretos, porque el dominio los pide distintos: la especificación escrita ocupa el lugar de los escenarios Gherkin, y los entornos de simulación ocupan el de la transformación automática de modelos a código.

El resto del artículo está organizado así. La sección II revisa los enfoques de referencia y justifica la sustitución. La sección III describe el diseño del modelo por sus seis componentes. La sección IV presenta el modelo de procesos completo y lo recorre Fase por Fase. La sección V recoge las conclusiones.

II. MARCO DE REFERENCIA

A. BDD y MDD como punto de partida

El desarrollo orientado al comportamiento, BDD, busca que el experto del negocio y el equipo técnico estén hablando del mismo sistema. Su instrumento es el escenario en lenguaje natural, escrito en un formato estándar como Gherkin, que después se convierte en prueba automatizada. Su virtud no es la prueba: es que la conversación queda escrita antes de programar.

El desarrollo orientado a modelos, MDD, parte de modelos que representan la lógica, la arquitectura y el comportamiento del sistema, los valida, y desde ellos genera artefactos ejecutables. Su virtud es que el modelo, y no el código, es la fuente de verdad.

Ambos objetivos se conservan en este trabajo. Lo que cambia son las herramientas con las que se persiguen.

B. Spec-Driven Development, en el lugar de BDD

Con el caficultor hay que acordar algo más duro que un caso de prueba. Hay que acordar restricciones que el sistema no puede violar bajo ninguna circunstancia: cuánta agua puede aplicar, en qué franja horaria, sobre qué lote, con qué margen de seguridad de vuelo. Un escenario Gherkin describe bien un comportamiento esperado, pero es un mal lugar para escribir una prohibición permanente.

Por eso el modelo adopta una cadena documental en la que la especificación manda sobre el código:

Constitution.md → spec.md → plan.md → task.md
→ código → verificación

Cada eslabón nace del anterior y el código es el último, nunca el primero. El Constitution.md es el documento que se redacta en la Fase 1 y que fija los principios y restricciones no

negociables del sistema; no se reescribe cada iteración, se audita en cada una. El efecto que se busca es el mismo de BDD —una especificación legible por quien no programa y anterior a la implementación— con la ventaja de que el artefacto resultante gobierna el proyecto entero y no solo un conjunto de casos.

La segunda mitad de BDD, la validación por parte de quien conoce el negocio, no solo se conserva sino que se lleva más lejos. En el modelo base el experto del dominio es un interesado al que se consulta. Aquí el caficultor es un Rol con Tareas propias: captura el conocimiento inicial del dominio, valida las reglas agronómicas, redacta el manual de operación y conduce el review de cada incremento. Y dos de las cuatro puertas de decisión del proceso las cierra él.

C. La simulación como modelo ejecutable

En un sistema ciberfísico la transformación automática de modelos a código rinde poco. Generar el firmware desde un diagrama ahorra unas horas de escritura, pero el riesgo del proyecto no está ahí: está en que la lógica de control se comporte de forma imprevista cuando la conecten a un actuador real, en una ladera, con un dron encima.

El valor del modelo, entonces, no es generar código: es poder ejecutar el sistema antes de que el hardware exista. Los modelos del proyecto son, por eso, entornos de ejecución:

- **SIL** (*software in the loop*) valida la lógica de control sin hardware, contra sensores y actuadores simulados.
- **HIL** (*hardware in the loop*) la valida con dispositivos reales dentro del bucle, sobre un banco de pruebas.
- El **gemelo digital** replica el terreno y sus dispositivos, y es donde cada incremento se verifica antes de bajar a campo.
- **MAPE-K** (*Monitor, Analyze, Plan, Execute, Knowledge*) es el bucle de adaptación sobre el que se construye la lógica de riego, y le da su forma al modelo de arquitectura.

De MDD se conservan la validación previa de los modelos y las pruebas automatizadas contra ellos. Se abandona la generación automática de código.

D. SPEM 2.0 como notación

El modelo se expresa en SPEM 2.0, el metamodelo de OMG para procesos de ingeniería de software y sistemas [2], en la composición que usa Eclipse Process Framework Composer. Se emplea el subconjunto mínimo necesario: Fase, Tarea, Rol, Producto de Trabajo, Hito, Actividad y Proceso. No se implementa la separación entre *Method Content* y *Process*: cada Fase declara sus propios Roles y Tareas, sin biblioteca reutilizable, porque el modelo describe un proyecto y no un método para muchos.

III. DISEÑO DEL MODELO DE PROCESOS

Siguiendo la taxonomía de modelos de procesos de métodos de diseño [3], que el trabajo base también emplea, el modelo se describe por seis componentes.

A. Ciclo

El proyecto tiene dos regímenes de trabajo distintos, y el modelo los separa en vez de forzarlos a uno solo.

El primero es la **Iteración 0**, que agrupa las Fases 1, 2 y 3 y ocurre una sola vez, entre las semanas 1 y 14. No es un sprint largo: es el arranque que produce la gobernanza, los contratos entre dominios, los entornos de simulación y el primer prototipo que atraviesa el sistema de extremo a extremo. Corre en cascada de manera deliberada, porque no tiene sentido iterar sobre una arquitectura que todavía no se ha escrito ni sobre unos contratos que aún no se han acordado.

El segundo es el **Ciclo de crecimiento**, la Fase 4, que es el estado permanente del proyecto a partir de la semana 15. Cada incremento dura de dos a cuatro semanas y recorre siempre el mismo trayecto: se sincroniza, se construye, se verifica, se despliega, se audita, se documenta y se muestra al caficultor. Lo aprendido en esa demostración reordena el backlog del incremento siguiente.

La duración del incremento es la primera diferencia con el trabajo base, que propone sprints de dos a tres semanas. Aquí el rango sube a cuatro porque instalar y calibrar dispositivos en un terreno real no cabe en dos.

B. Colaboración

El equipo son ocho Roles. Cinco pertenecen al software y a la gestión: el Gerente de proyecto, el Ingeniero de datos, el Ingeniero de software de adaptación, el Ingeniero de seguridad de vuelo y el QA de software. Dos pertenecen al mundo físico: el Ingeniero electrónico y el Ingeniero mecatrónico. El octavo es el Experto del dominio, es decir, el caficultor.

Aquí está la segunda diferencia con el trabajo base. Aquel agrupa todos los perfiles en un *Agile Team* y trata al desarrollador de dispositivos IoT como un rol más dentro de ese grupo. En este modelo la electrónica y la mecatrónica son trabajo de ingeniería con entidad propia: tienen Tareas, producen artefactos y montan un banco de pruebas de hardware que existe antes que el software destinado a comandarlo. Y el caficultor, como ya se dijo, tiene Tareas en vez de una silla en la reunión.

C. Artefactos

Los Productos de Trabajo se agrupan en cuatro familias. Los de **gobernanza**: el *Constitution.md*, los contratos globales y de integración entre dominios, las políticas y límites de operación y las reglas de negocio. Los de **especificación**: *spec.md*, *plan.md*, *task.md*, el modelo de arquitectura y las especificaciones de la lógica de adaptación. Los del **mundo físico**: los esquemáticos con su lista de materiales, el banco de pruebas de hardware, el banco HIL, la red de sensores instalada, el firmware y el paquete de calibración. Y los de **producto y aprendizaje**: el prototipo vertical, los informes de validación de control, el pipeline de CI/CD, los releases, el manual de operación y las lecciones que se llevan al backlog.

D. Uso recomendado

Proyectos pequeños a medianos, con equipos de cinco a diez personas. El riesgo técnico se estima **alto**, por encima del que asigna el trabajo base a su propuesta, porque el sistema no falla solamente por software: falla por un sensor descalibrado, por un dron que pierde enlace o por una regla agronómica que en el papel funcionaba y en la ladera no.

E. Madurez

Media. SPEM 2.0 y el trabajo por incrementos están ampliamente difundidos y documentados; la combinación concreta de una cadena de especificación escrita con validación en SIL y HIL es más reciente. El modelo se apoya en lenguaje natural para que cualquier participante lo entienda sin formación previa en la notación, que es también la razón por la que las figuras llevan leyenda.

F. Flexibilidad

Alta en el orden interno de las Tareas y en el contenido de cada incremento, que se decide en la sincronización que abre el ciclo. Baja en las cuatro puertas de decisión, que son condiciones de avance y no acuerdos revisables. La granularidad es de un documento principal por ítem, ampliado por documentos satélite cuando hace falta.

La tabla I resume las cuatro sustituciones respecto del trabajo base.

Tabla I
QUÉ SE CONSERVA Y QUÉ SE SUSTITUYE

EJE	TRABAJO BASE	ESTA PROPUESTA	MOTIVO
Lenguaje	Escenarios Gherkin	Especificación escrita	Se acuerdan restricciones, no casos
Experto	Interesado consultado	Rol con Tareas y puertas	Su criterio condiciona el avance
Modelos	MDD y generación de código	SIL, HIL y gemelo digital	El riesgo vive en el mundo físico
Ciclo	Sprints de 2-3 semanas	Incrementos de 2-4 semanas	Calibrar en campo no cabe en dos

IV. PROPUESTA DEL MODELO DE PROCESOS

El modelo completo son cuatro Fases, veintiuna Tareas y ocho Roles, dibujados en una sola red en la figura 1. Antes de recorrerlo conviene explicar cómo se lee, porque la figura está construida para poder contarse sola.

A. Cómo se lee el modelo consolidado

Cada Fase es una columna que se lee hacia abajo, y las Fases se suceden hacia la derecha. Sobre el canal que une una columna con la siguiente se apilan los Productos de Trabajo del traspaso, porque la Salida de una Fase es la Entrada de la que sigue: ahí es donde se ve qué se hereda y qué se descarta al cambiar de etapa.

Cada celda es una Tarea y tiene tres bandas. Arriba van los Roles que participan, y la tipografía distingue el papel de cada uno: en negrita el Rol que ejecuta la Tarea y responde por lo que sale de ella, en trazo normal los que asisten. En el centro va la Tarea con su código —T1.1, T2.6, T4.3— que es lo que permite referirse a ella desde el texto y desde las tablas. Abajo van los Productos de Trabajo que la Tarea produce.

Los rombos son puertas de decisión y ninguna es decorativa: cada una lleva dibujada su rama negativa hasta la Tarea concreta donde se rehace el trabajo. La elipse que ocupa la parte inferior es la Fase 4, y el hecho de que sea un anillo y no una columna es la afirmación central del modelo: esa Fase no termina, gira.

La figura lleva además dos elementos que SPEM 2.0 no modela y que sin embargo el proyecto necesita: una regla de tiempo en semanas sobre las columnas, y un plan de releases al pie. Un modelo de procesos dice cómo se trabaja; sin esas dos capas no dice cuándo se entrega nada.

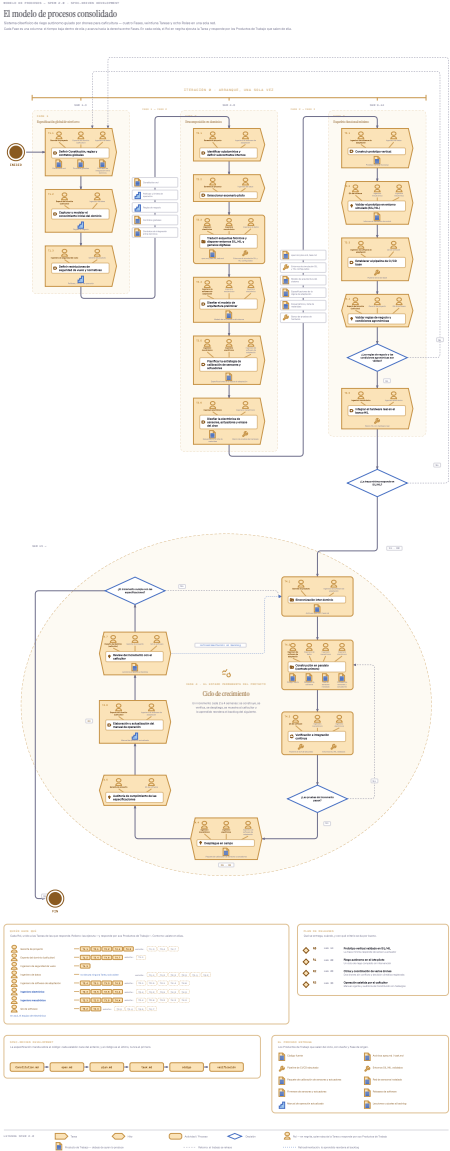


Fig. 1. Modelo de procesos consolidado. Cada Fase es una columna que se lee hacia abajo; las Fases se suceden hacia la derecha y sobre el canal que las une van los Productos de Trabajo del traspaso. En cada celda, los Roles arriba —en negrita el que ejecuta la Tarea—, la Tarea al centro con su código y sus Productos de Trabajo abajo. Los rombos son puertas de decisión, con su retorno dibujado. La elipse inferior es la Fase 4, que gira mientras el proyecto viva. Es una figura de conjunto: las figuras 2 a 5 la amplían Fase por Fase.

B. Inicio y Fase 1: especificación global de nivel cero

El proceso comienza, igual que en el trabajo base, con una reunión. Pero no entre un analista de negocio y unos interesados, sino entre el Gerente de proyecto y el caficulator. De ese encuentro no sale un backlog: sale el Constitution.md, el documento que fija lo que el sistema no puede violar por mucho que cambie el alcance. El Gerente de proyecto ejecuta esa

Tarea, T1.1, con la asistencia del caficutor y del Ingeniero de datos, partiendo de la normatividad vigente, y de ella salen además los contratos globales y los contratos de integración entre dominios que harán posible, mucho después, que tres equipos escriban a la vez sin bloquearse.

En paralelo el caficutor ejecuta T1.2, donde su conocimiento y los planos del terreno se convierten en reglas de negocio escritas; lo asiste el Ingeniero de datos, que es quien tiene que dejarlas en una forma que un sistema pueda evaluar. Y el Ingeniero de seguridad de vuelo ejecuta T1.3, con la asistencia del Gerente de proyecto, para delimitar dónde y cuándo puede volar un dron: de ahí salen las políticas y límites de operación, que son la otra restricción dura del proyecto.

La Fase entrega, entonces, cinco Productos de Trabajo: la Constitución, las políticas y límites de operación, las reglas de negocio y los dos juegos de contratos. Estos procesos se muestran en la figura 2.

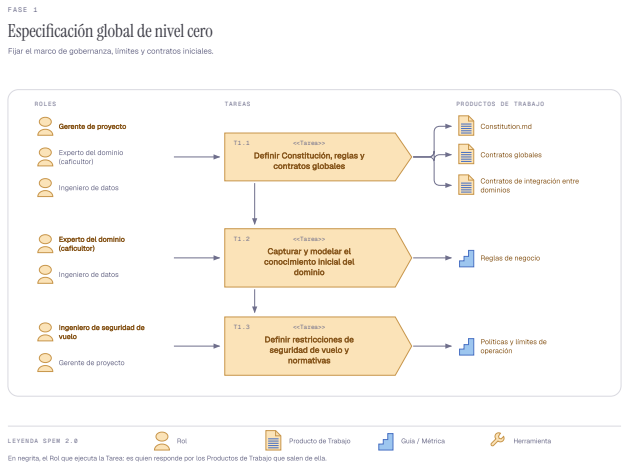


Fig. 2. Fase 1. Los Roles a la izquierda, la cadena de Tareas al centro y los Productos de Trabajo que salen de cada una a la derecha. En negrita, el Rol que ejecuta la Tarea y responde por sus artefactos.

C. Fase 2: descomposición en dominios

Con los contratos acordados, la Fase 2 parte el sistema en subdominios y levanta el andamiaje que permitirá construir sin hardware. El Gerente de proyecto ejecuta las dos Tareas de encuadre: T2.1, donde define los subcontratos internos entre datos, adaptación, vuelo y electrónica, y T2.2, donde escoge el lote piloto sobre el que se probará todo.

El resto de la Fase avanza en paralelo, y es aquí donde el modelo se separa más del trabajo base. El equipo de electrónica y mecatrónica ejecuta T2.3, que dispone los entornos SIL, HIL y el gemelo digital y produce además los tres archivos de la cadena de especificación: spec.md, plan.md y task.md. El Ingeniero de software de adaptación ejecuta T2.4 y diseña la arquitectura preliminar sobre el bucle MAPE-K, asistido por los dos ingenieros de hardware, porque una arquitectura de control que ignore las latencias reales del actuador no sirve. Esos mismos dos ingenieros ejecutan T2.5, la estrategia de calibración, y T2.6, el diseño de la electrónica de sensores, actuadores y enlace del dron, de donde salen los esquemáticos, la lista de materiales y el banco de pruebas de hardware.

Al terminar la Fase existe todo lo necesario para programar contra un sistema que todavía no se ha construido: especificaciones, arquitectura, entornos de simulación y un banco físico donde probar. Estos procesos se muestran en la figura 3.

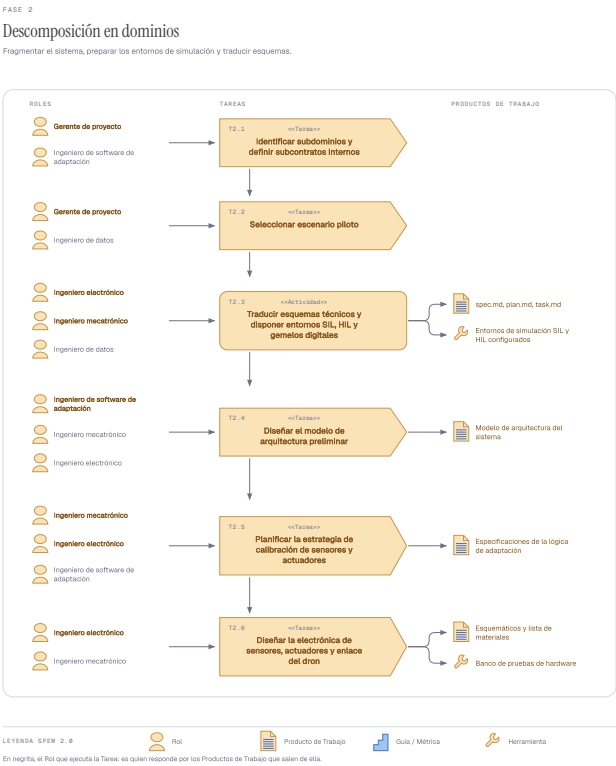


Fig. 3. Fase 2. Seis Tareas que avanzan más en paralelo que en cadena: la arquitectura, los entornos de simulación y el diseño de la electrónica progresan a la vez, atados por los contratos acordados en la Fase 1.

D. Fase 3: esqueleto funcional mínimo

La Fase 3 no busca funcionalidad. Busca una sola traza que atraviese el sistema completo: un sensor que lee, una regla del MAPE-K que decide y un actuador que responde. El Ingeniero de software de adaptación la construye en T3.1, asistido por el Ingeniero de datos, y el QA la valida en T3.2 dentro de los entornos simulados, con el acompañamiento de los dos ingenieros de hardware, produciendo los informes de validación de control. En T3.3 se automatiza el empaquetado en un pipeline de CI/CD base.

Antes de montar hardware, sin embargo, hay una puerta. En T3.4 el caficutor valida las reglas de negocio y las condiciones agronómicas, asistido por el Gerente de proyecto y el QA. Lo que él no valida no se corrige programando: se corrige volviendo a la Fase 1 a reescribir la regla, y así está dibujado el retorno. Superada esa puerta, el Ingeniero mecatrónico ejecuta T3.5 y monta sensores y actuadores reales contra la lógica de control. Es el punto donde el equipo de software y el de electrónica se encuentran por primera vez, y el resultado es el banco HIL con hardware real.

La Fase cierra con la segunda puerta, que pregunta si la traza mínima responde en SIL y HIL. Si responde, se corta el release R0 y el proyecto entra en su ciclo permanente. Si no

responde, el retorno tampoco es a la Tarea anterior: vuelve a la Fase 1, a revisar la Constitución, las reglas y los contratos que la sostienen. Estos procesos se muestran en la figura 4.

FASE 3

Esqueleto funcional mínimo

Construir el prototipo vertical: unir extremo a extremo una traza mínima del sistema.

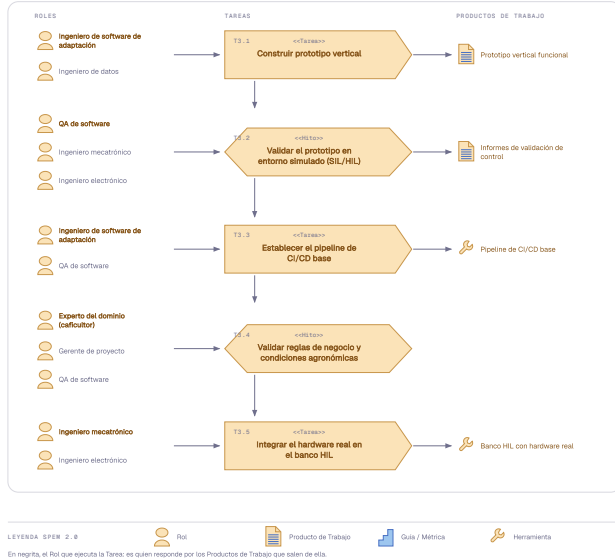


Fig. 4. Fase 3. Cinco Tareas para construir y validar una sola traza. Los dos Hitos —la validación en SIL/HIL y la validación agronómica del caficultor— son las puertas que deciden si la Iteración 0 termina.

E. Fase 4: ciclo de crecimiento

La Fase 4 es el estado permanente del proyecto y por eso se dibuja como un anillo. Cada vuelta es un incremento y recorre siempre el mismo trayecto.

Abre el Gerente de proyecto con T4.1, la sincronización inter dominio, donde se acuerda qué entra en el incremento y se actualizan los archivos spec.md y task.md. Sigue T4.2, la construcción en paralelo, que ejecuta el Ingeniero de software de adaptación con la asistencia del Ingeniero de datos y de los dos de hardware. El paralelismo que el trabajo base introduce entre el desarrollador de software y el de dispositivos aparece exactamente aquí, y funciona por una razón concreta: los contratos de integración firmados en la Fase 1 permiten que los tres frentes escriban al mismo tiempo sin esperarse. Del mismo incremento salen, a la vez, el código, los releases, la red de sensores instalada y el firmware.

El QA ejecuta después T4.3, la verificación e integración continua sobre el pipeline y el banco HIL, y ahí está la tercera puerta: si las pruebas no pasan, el incremento vuelve a construcción. Si pasan, baja a campo. El despliegue, T4.4, lo ejecutan los ingenieros mecánico y electrónico, que instalan y calibran los dispositivos y producen el paquete de calibración; es el corte de los releases R1 y R2.

Las tres Tareas que cierran la vuelta son las que convierten un incremento técnico en un incremento entregado. En T4.5 el Gerente de proyecto audita que el incremento no haya roto la Constitución ni se haya salido del plan acordado. En T4.6 el caficultor actualiza el manual de operación, que es lo que corta

el release R3, porque el sistema no está entregado cuando funciona sino cuando él puede operarlo. Y en T4.7 el caficultor conduce el review del incremento en su propio terreno.

De esa última Tarea salen las lecciones y ajustes al backlog, y aquí conviene distinguir dos retornos que la figura dibuja distinto a propósito. Uno es la cuarta puerta: si el incremento no cumple las especificaciones, vuelve a la sincronización, que es donde se decide qué se rehace y con qué prioridad. El otro es la retroalimentación del review, que también llega a la sincronización pero no porque algo haya fallado, sino porque lo aprendido en el terreno reordena el backlog del incremento siguiente. Es la retroalimentación continua que el trabajo base atribuye a BDD, puesta aquí como una arista propia del modelo. Estos procesos se muestran en la figura 5.

FASE 4

Ciclo de crecimiento

Escalar el sistema mediante desarrollo paralelo, integración continua y validación en el cultivo.

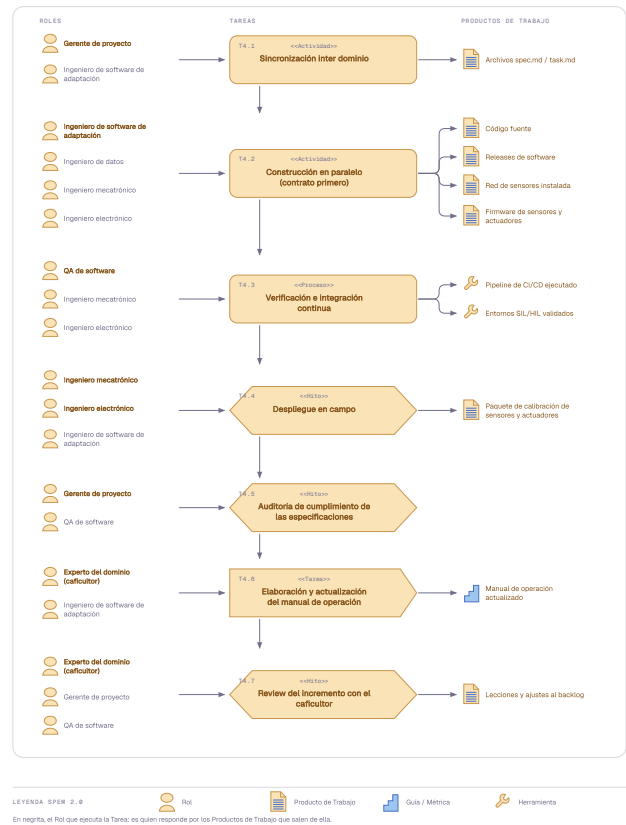


Fig. 5. Fase 4. El incremento completo: siete Tareas, cuatro Hitos y el caficultor ejecutando tres de ellos. Es la vuelta que el proyecto repite cada dos a cuatro semanas.

F. Las puertas de decisión

Ninguna Fase de este modelo avanza porque se le haya acabado el tiempo asignado. Avanza porque pasó una puerta. Son cuatro y están resumidas en la tabla II, con el retorno que cada una activa cuando la respuesta es negativa.

Las dos primeras retornan a la Fase 1 y no a la Tarea inmediatamente anterior, lo que a primera vista parece excesivo. Es deliberado: en un sistema ciberfísico, un prototipo que no responde rara vez se arregla reescribiendo el prototipo. La com-

paración con el trabajo base es directa. Allí un defecto crítico detiene el despliegue y abre una tarea de mantenimiento; aquí un fallo de la traza mínima se interpreta como un defecto de la especificación que la produjo, y se corrige donde se originó.

Tabla II
LAS CUATRO PUERTAS DE DECISIÓN

PUERTA	DÓNDE	SI LA RESPUESTA ES NO
¿Las reglas de negocio y las condiciones agronómicas son válidas?	Fase 3, tras T3.4	Vuelve a la Fase 1: la regla está mal escrita, no el código
¿La traza mínima responde en SIL/HIL?	Cierre de la Fase 3	Vuelve a la Fase 1, a revisar Constitución, reglas y contratos
¿Las pruebas del incremento pasan?	Fase 4, tras T4.3	Vuelve a T4.2, construcción en paralelo
¿El incremento cumple con las especificaciones?	Fase 4, tras T4.5	Vuelve a T4.1, sincronización inter dominio

G. Plan de releases

SPEM 2.0 modela el proceso, no el producto. Para que el modelo diga cuándo se entrega algo hay que añadir la dimensión de producto de forma explícita, y eso hace la tabla III. Cada release lleva su criterio de aceptación, redactado como un hecho observable y no como un porcentaje de avance.

Tabla III
PLAN DE RELEASES

REL.	SEM	QUÉ SE ENTREGA	CRITERIO DE ACEPTACIÓN
R0	14	Prototipo vertical validado en SIL/HIL	La traza mínima responde de sensor a actuador
R1	20	Riego autónomo en el lote piloto	Un ciclo de riego completo sin intervención
R2	28	Clima y coordinación de varios drones	Dos drones sin conflicto y decisión climática registrada
R3	36	Operación asistida por el caficultor	Manual vigente y auditoría de Constitución sin hallazgos

R0 lo corta la puerta de salida de la Fase 3, R1 y R2 el despliegue en campo y R3 el manual de operación.

H. Reparto de responsabilidades

La tabla IV recoge, para cada Rol, las Tareas que ejecuta —y de cuyos Productos de Trabajo responde— y aquellas en las que asiste. Es la lectura por persona de lo que la figura 1 muestra por Fase.

Tabla IV
QUÉ EJECUTA Y QUÉ ASISTE CADA ROL

ROL	EJECUTA	ASISTE
Gerente de proyecto	T1.1 T2.1 T2.2 T4.1 T4.5	T1.3 T3.4 T4.7
Experto del dominio (caficultor)	T1.2 T3.4 T4.6 T4.7	T1.1
Ing. de seguridad de vuelo	T1.3	—
Ing. de datos	—	T1.1 T1.2 T2.2 T2.3 T3.1 T4.2
Ing. de software de adaptación	T2.4 T3.1 T3.3 T4.2	T2.1 T2.5 T4.1 T4.4 T4.6
Ing. electrónico	T2.3 T2.5 T2.6 T4.4	T2.4 T3.2 T3.5 T4.2 T4.3
Ing. mecatrónico	T2.3 T2.5 T3.5 T4.4	T2.4 T2.6 T3.2 T4.2 T4.3
QA de software	T3.2 T4.3	T3.3 T3.4 T4.5 T4.7

La tabla deja a la vista dos asimetrías que conviene resolver antes de dar el modelo por cerrado. El Ingeniero de datos no ejecuta ninguna Tarea: asiste en seis, repartidas por las cuatro Fases, pero no responde por ningún Producto de Trabajo. O el dato es una preocupación transversal sin dueño único, o falta en el modelo una Tarea de modelado del dato del cultivo que debería ser suya. El Ingeniero de seguridad de vuelo, por su parte, aparece en una sola Tarea y luego desaparece; sus políticas y límites de operación, en cambio, alimentan tres Tareas de la Fase 2, de modo que su trabajo se concentra al inicio y gobierna el resto por escrito.

V. CONCLUSIONES

Los proyectos ciberfísicos requieren un equipo multidisciplinar, y este modelo lo hace explícito en su estructura: tres de los ocho Roles pertenecen al mundo físico y tienen Tareas y artefactos propios, en vez de figurar como apéndice del equipo de software. La construcción de hardware deja de ser un supuesto y pasa a ocupar espacio en el cronograma.

La comunicación entre los participantes se gestiona con artefactos y no con reuniones. La cadena que va de la Constitución al código cumple la función que el trabajo base asigna a BDD —dejar el acuerdo escrito antes de programar— y agrega una capa de gobernanza, porque el documento inicial no caduca: se audita en cada incremento.

La validación anticipada sustituye con ventaja a la generación automática de código. SIL, HIL y el gemelo digital permiten ejecutar el sistema antes de que el hardware exista, que es exactamente donde vive el riesgo del proyecto, mientras que la generación de código habría ahorrado un esfuerzo que no era el cuello de botella.

El paralelismo entre equipos, que el trabajo base propone como característica del modelo, se sostiene aquí sobre una condición previa: los contratos de integración acordados en la Fase 1. Sin ese acuerdo escrito, tres frentes trabajando a la vez no se aceleran, se bloquean.

Por último, la participación del experto del dominio deja de ser consultiva. El caficultor ejecuta cuatro Tareas y cierra dos de las cuatro puertas del proceso, y dos de los retornos del modelo terminan en la Fase 1 porque una regla agronómica

mal escrita se considera un defecto de especificación y no de implementación. En un sistema que decide cuándo regar un cultivo, esa distinción es la que evita automatizar un error.

REFERENCIAS

1. Metodología con enfoque BDD y MDD para el desarrollo de proyectos IoT. Documento base de este trabajo.
2. OMG, «Software & Systems Process Engineering Metamodel Specification (SPEM) 2.0». [En línea]. Disponible en: <https://www.omg.org/spec/SPEM>
3. E. Céret, S. Dupuy-Chessa, G. Calvary, A. Front y D. Rieu, «A Taxonomy of Design Methods Process Models», *Information and Software Technology*, vol. 55, pp. 795–821, 2013.
4. Eclipse Foundation, «Eclipse Process Framework (EPF) Composer». Notación empleada en las figuras de este documento.
5. J. O. Kephart y D. M. Chess, «The Vision of Autonomic Computing», *Computer*, vol. 36, n.º 1, pp. 41–50, 2003. Origen del bucle MAPE-K.
6. H. M. Abushama, H. Altigani y F. A. Elhaj, «The effect of Test-Driven Development and Behavior-Driven Development on Project Success Factors», *ICCCEEE*, 2020, pp. 2–7.